CasSecretTool — Encrypting Configuration Secrets

Operator manual for deployment / operations staff

System: Collateral Appraisal System API · **Generated:** 2026-07-29

Audience: deployment / operations staff on the app servers.
Purpose: replace every plaintext password in `appsettings.Production.json` with an encrypted `ENC:v1:…` value, so no credential sits in plaintext on disk.

This manual is a standalone how-to. It covers importing the certificate (via the Windows UI), encrypting each secret, and verifying the result. Background on the design is in `multi-server-deployment.md` **§2.12**.

---

1. What the tool does

`CasSecretTool` encrypts a secret value so it can be pasted into `appsettings.Production.json` as `ENC:v1:<base64>`. The application decrypts it automatically at startup using a certificate in the Windows certificate store. The tool uses the *same code* the application uses to decrypt, so a value it produces is guaranteed readable by the app.

It has two actions:

Action	What it does
encrypt	Turns a plaintext password into an `ENC:v1:…` value to paste into config.
verify	Decrypts an `ENC:v1:…` value and shows a **masked** confirmation, to check a value is correct *before* restarting the app.

The tool **never writes anything to disk** and never logs the secret.

Before you start, make sure:

1. You have the `cas-secrets.pfx` file **and** its password (from your secrets vault). The certificate itself is generated once — see **Appendix A** — but on each server your job is just to **import** it (§2).
2. The tool is present under the release folder: `C:\Deploy\temp\<version>\tools\`.

---

2. Import the secrets certificate — via the Windows UI

Do this **once on every app server**. It has three parts, all done through the Windows UI: import the certificate, grant the app pool access to its private key, and read its thumbprint. (Prefer PowerShell? See **Appendix B**.)

> ⚠ **Critical:** the certificate must land in **Local Machine** → **Personal**, *not* Current User. The app runs as the IIS app pool and can only read the Local Machine store. Choosing the wrong store is the #1 cause of "app won't start after deploy".

2.1 Import the PFX (Certificate Import Wizard)

1. Copy `cas-secrets.pfx` onto the server (e.g. `C:\Deploy\temp\<version>\`).
2. **Double-click** `cas-secrets.pfx`. The **Certificate Import Wizard** opens.
3. **Store Location:** select **Local Machine** → **Next**. (Accept the UAC prompt — this needs administrator rights. If you don't see a "Local Machine" option you double-clicked as a standard user; use the `certlm.msc` route below instead.)
4. **File to import:** confirm the path shows `cas-secrets.pfx` → **Next**.
5. **Private key protection:**
   - Enter the **PFX password**.
   - Leave **Mark this key as exportable** *unchecked* (the key should not leave this server).
   - Tick **Include all extended properties**.
   - → **Next**.

6. **Certificate Store:** choose **Place all certificates in the following store**, click **Browse…**, select **Personal**, **OK** → **Next**.
7. **Finish**. You should see *"The import was successful."*

Alternative — via `certlm.msc`: press `Win`+`R`, type `certlm.msc`, `Enter` → **Personal** → **Certificates** → right-click → **All Tasks** → **Import…** → the same wizard runs, already scoped to Local Machine.

2.2 Grant the app pool read access to the private key

Without this the app starts but fails with *"has no accessible private key"*.

1. Open `certlm.msc` (**Win+R** → `certlm.msc`).
2. Go to **Personal** → **Certificates**.
3. Right-click **CollateralAppraisal-Secrets** → **All Tasks** → **Manage Private Keys…**.
4. Click **Add…**, type the app pool identity `IIS AppPool\<YourAppPool>` (e.g. `IIS AppPool\CAS`), click **Check Names**, then **OK**.
5. Select that identity, tick **Read** under *Allow*, click **Apply / OK**.

> The same certificate (with its private key) and this **Read** grant must exist on **every** app server behind the load balancer.

2.3 Read the thumbprint

You need the thumbprint for `appsettings.Production.json` (§7).

1. In `certlm.msc`, **Personal** → **Certificates**, **double-click** `CollateralAppraisal-Secrets`.
2. Open the **Details** tab, scroll to **Thumbprint**, and click it.
3. Copy the 40-character hex value from the box below.

> ⚠ Paste it into a plain-text editor first and **remove any spaces** (and a possible invisible character Windows prepends). `A1 B2 C3…` must become `A1B2C3…`, or the app won't find the cert.

2.4 Confirm it's installed

In `certlm.msc` → **Personal** → **Certificates**, the entry `CollateralAppraisal-Secrets` should be listed **with a small key icon** on it — that icon means the private key is present. If the icon is missing, the PFX was imported without its key; delete it and redo §2.1.

---

3. Which certificate to select — IMPORTANT

When the tool lists certificates, **select the one whose thumbprint matches `Secrets:CertificateThumbprint` in `appsettings.Production.json`** — normally `CN=CollateralAppraisal-Secrets`.

```
Certificates with a private key:
  [1] CN=CollateralAppraisal-Secrets     (A1B2C3…)   ← SELECT THIS
  [2] CN=CollateralAppraisal-Signing     (D4E5F6…)   ← do NOT use (OAuth2 JWT signing)
  [3] CN=CollateralAppraisal-Encryption  (E7F8A9…)   ← do NOT use (OAuth2 JWT encryption)
```

Why it matters: the app decrypts with exactly one certificate — the one named in the config. If you encrypt a value with a *different* certificate, the app will **fail to start** with a `Failed to decrypt configuration value '<key>'` error. Encrypting always appears to succeed (encryption only needs the public key); the mismatch is only caught at startup — which is why **§6 (verify) is mandatory**.

Do **not** use the OAuth2 signing/encryption certificates: they rotate on a different schedule, and reusing them would make every secret undecryptable the day those certs are rotated.

---

4. Encrypt a secret (interactive — recommended)

On the app server:

```
cd C:\Deploy\temp\<version>\tools
.\CasSecretTool.exe
```

Follow the prompts:

```
CAS Secret Tool
===============
Certificates with a private key:
  [1] CN=CollateralAppraisal-Secrets  (A1B2C3…)
[LocalMachine]
Select cert: 1
(e)ncrypt or (v)erify: e
Value: **********           ←  type/paste the real
password; it is NOT shown on screen
ENC:v1:AAELc4zuwOA/ql4O...  ←  copy this ENTIRE line
```

Copy the whole `ENC:v1:…` line into the matching field in `appsettings.Production.json`:

```
"Mail": {
  "Password": "ENC:v1:AAELc4zuwOA/ql4O..."
}
```

Repeat for every secret in the checklist (§8).

---

5. Encrypt a secret (scriptable — optional)

If you prefer flags (e.g. inside a script):

```
# Prompts for the value (not echoed); prints the ENC:v1:
result to stdout.
.\CasSecretTool.exe protect --thumbprint A1B2C3D4E5F6...

# Or pipe the value in (the prompt goes to stderr, so
stdout is only the result):
"P@ssw0rd" | .\CasSecretTool.exe protect --thumbprint
A1B2C3D4E5F6...
```

---

6. Verify a value BEFORE restarting the app — mandatory

After pasting a value into config, confirm it decrypts. This runs the **exact** decrypt path the application uses, so a passing verify guarantees the app can read it.

```
.\CasSecretTool.exe verify --thumbprint A1B2C3D4E5F6... --
value "ENC:v1:AAELc4zu..."
```

Expected output:

```
OK — decrypts successfully to: P@s****
```

- The confirmation is **masked** (first 3 chars + `****`) — enough to recognise the password without showing it on screen.
- If verify **fails**, you selected the wrong certificate, or the value was truncated when pasted. Re-encrypt with the correct certificate. Do **not** restart the app until verify passes.

You can also verify interactively: run `.\CasSecretTool.exe`, pick the cert, choose `v`, and paste the value.

---

7. Point the application at the certificate

Set the thumbprint in `appsettings.Production.json` (the deployment template already has the token):

```
"Secrets": {
  "CertificateThumbprint": "A1B2C3D4E5F6..."
}
```

If this is left blank, the app falls back to `DataProtection:CertificateThumbprint`. Strip any spaces or invisible characters copied from `certlm.msc` (see §2.3).

---

8. Which values must be encrypted

Encrypt every value below that is present and non-empty:

- [ ] `ConnectionStrings:Database` (the whole string — it contains `Password=…`)
- [ ] `ConnectionStrings:Hangfire` (same)
- [ ] `RabbitMQ:Password`
- [ ] `Mail:Password`

- [ ] `Ldap:BindPassword` (only if a service-account bind is used)
- [ ] `SeedData:AdminUser:Password`
- [ ] `FileTransfer:Inbound:Sftp:Password`
- [ ] `FileTransfer:Outbound:Sftp:Password`

> `ConnectionStrings:Redis` has no password by default — encrypt it only if yours does.

At startup, the application logs an **error** naming any of these keys still left in plaintext (the value is never logged), so a missed one is easy to spot in the logs.

---

9. Final check on the server

Confirm no plaintext password remains — this should return **no output**:

```
Select-String -Path appsettings.Production.json -Pattern
'Password' |
    Select-String 'ENC:v1:' -NotMatch
```

Then start the application and confirm it boots. A wrong certificate or a corrupted value fails fast at startup, naming the offending key (never its value).

Repeat this on **every** app server — the same certificate (same thumbprint) must be installed on all of them.

---

10. Rotating the certificate

There is no dual-key transition, so keep the window short:

1. Install the new secrets certificate on **all** app servers (generate per **Appendix A**, import per **§2**).
2. Re-encrypt **every** `ENC:v1:` value with the new certificate using this tool.
3. Update `Secrets:CertificateThumbprint` to the new thumbprint.

4. Restart the application on each node.

Add the new certificate's expiry to the same monitoring that covers the OAuth2 certificates.

---

11. Troubleshooting

Symptom	Cause / fix
`No certificates with a private key found`	Secrets cert not icon per §2.4).
`Certificate with thumbprint '…' was not found`	Wrong thumbpri Certificates.
App start: `Failed to decrypt configuration value '<key>'`	The value was e `Secrets:Cert`
`…has no accessible private key`	The app pool ide
verify shows the wrong password	You encrypted th
`CasSecretTool.exe` is not in the `tools\` folder	The artifact was `dotnet CasSe` `dotnet CasSe`

---

12. Security notes

- The tool reads the value **without echoing** it and prints prompts to stderr so a piped result cannot leak the secret into a capture.
- No secret value is ever written to disk or logged — by the tool or by the application.
- Losing the certificate means the encrypted values are **unrecoverable**. Keep the original plaintext values **and** the certificate PFX (with its password) in the bank's secrets vault.

Appendix A — Generate the secrets certificate (once)

Done **once**, by whoever provisions certificates, on a build/admin workstation. There is no practical UI to create a self-signed certificate with the required key usage, so this step uses PowerShell (run as Administrator). The operator on each server only needs the resulting `cas-secrets.pfx` file and its password — importing it is the UI process in **§2**.

```
# Secrets certificate — RSA-2048, KeyEncipherment +
DataEncipherment, 10-year life.
$secCert = New-SelfSignedCertificate `
  -Subject       "CN=CollateralAppraisal-Secrets" `
  -KeyAlgorithm  RSA `
  -KeyLength     2048 `
  -HashAlgorithm SHA256 `
  -KeyUsage      KeyEncipherment, DataEncipherment `
  -NotAfter      (Get-Date).AddYears(10) `
  -CertStoreLocation Cert:\CurrentUser\My

$secPwd = ConvertTo-SecureString -String "
<SECRETS_PFX_PASSWORD>" -AsPlainText -Force
Export-PfxCertificate -Cert $secCert -FilePath .\cas-
secrets.pfx -Password $secPwd
Write-Host "Secrets thumbprint: $($secCert.Thumbprint)"

# Remove from the build box's user store — keep only the
.pfx + thumbprint.
Remove-Item "Cert:\CurrentUser\My\$($secCert.Thumbprint)"
```

Store `cas-secrets.pfx` **and** its password in the bank's secrets vault. This must be a **separate** certificate from the OAuth2 signing/encryption certs so it can be rotated independently.

Appendix B — Scripted import + grant access (PowerShell alternative to §2)

For teams that prefer scripting over the UI. Run as Administrator **on each app server**.

```
# 1. Import the PFX into LocalMachine\My (with private key)
$secPwd = ConvertTo-SecureString -String "
<SECRETS_PFX_PASSWORD>" -AsPlainText -Force
Import-PfxCertificate -FilePath .\cas-secrets.pfx `
  -CertStoreLocation Cert:\LocalMachine\My -Password
$secPwd

# 2. Grant the app pool read access to the private key
function Grant-CertReadAccess {
    param([string]$Thumbprint, [string]$AppPoolIdentity)
    $cert    = Get-ChildItem
"Cert:\LocalMachine\My\$Thumbprint"
    $keyName =
([System.Security.Cryptography.X509Certificates.RSACertificate
    $keyPath =
"$env:ProgramData\Microsoft\Crypto\Keys\$keyName"
    $acl     = Get-Acl $keyPath
    $rule    = New-Object
System.Security.AccessControl.FileSystemAccessRule($AppPoolIde
"Read", "Allow")
    $acl.AddAccessRule($rule); Set-Acl -Path $keyPath -
AclObject $acl
    Write-Host "Granted Read to $AppPoolIdentity"
}
Grant-CertReadAccess -Thumbprint "<secrets-thumbprint>" -
AppPoolIdentity "IIS AppPool\CAS"
```

This is the same operation as the GUI steps in §2.1–2.2; use whichever your team prefers.